AArch64 Memory & Cache System — Tchnical Reference Workbook 1
Chapter 10 — Complete CortexA72 CacheMaintenance Cookbook: All Patterns, All UseCases, All Sequences (Based on ARM64 CortexA72) 1
10.1 Scope 1
10.2 Core Principles (Summary) 1
10.2.1 Instruction Visibility 1
10.2.2 Data Visibility 2
10.2.3 DMA → CPU 2
10.2.4 MMIO 2
10.3 InstructionRelated Patterns 3
10.3.1 SelfModifying Code / JIT Engines 3
10.3.2 Bootloader Patching Code 3
10.3.3 Hypervisor Installing Guest Trampolines 3
10.3.4 Kernel Modifying Exception Vectors 3
10.3.5 CPU0 Preparing Code for CPU1 (SMP BringUp) 3
10.4 DataRelated Patterns 4
10.4.1 CPU → DMA (Device Reads CPU Data) 4
10.4.2 DMA → CPU (Device Writes Data) 4
10.4.3 CPU → Other Core (InterCore Data Visibility) 4
10.4.4 CPU → Other Cluster 5
10.4.5 Zeroing Memory Efficiently 5
10.5 MMIO / Device Memory Patterns 5
10.5.1 Writing Device Registers 5
10.5.2 Reading Device Registers 5
10.5.3 Writing DMA Descriptors 5
10.5.4 Reading DMA Completion Status 6
10.6 SMP Patterns 6
10.6.1 CPU0 → CPU1 Instruction Visibility 6
10.6.2 CPU0 → CPU1 Data Visibility 6

10.6.3 Secondary Core Boot Sequence 6
10.7 Hypervisor / Kernel Patterns 7
10.7.1 Installing Stage2 Page Tables 7
10.7.2 Modifying Stage1 Page Tables 7
10.7.3 Updating EL2 Vectors 7
10.7.4 Updating EL1 Vectors 8
10.8 Bootloader Patterns 8
10.8.1 MMU Enable Sequence 8
10.8.2 Cache Enable Sequence 8
10.8.3 Patching Boot Code 8
10.9 Complete Cookbook Table 8
10.10 Exercises (Technical Reference) 9
Exercise 10.1 — Instruction Visibility 9
Exercise 10.2 — DMA Read Path 9
Exercise 10.3 — DMA Write Path 9
Exercise 10.4 — MMIO Ordering 9
Exercise 10.5 — SMP BringUp 9

AArch64 Memory & Cache System — Tchnical Reference Workbook

Chapter 10 — Complete Cortex-A72 Cache-Maintenance Cookbook: All Patterns, All Use-Cases, All Sequences (Based on ARM64 Cortex-A72)

10.1 Scope

This chapter consolidates all cache-maintenance operations into a single, unified reference. It provides:

- All DC/IC instruction sequences
- All visibility patterns (PoU, PoC)
- All SMP patterns
- All DMA patterns
- All instruction-patching patterns
- All MMIO patterns
- All bootloader patterns
- All hypervisor/kernel patterns

This chapter is intended to be used as a **cookbook**: select the scenario, apply the exact sequence, and guarantee architectural correctness on ARM64 Cortex-A72.

10.2 Core Principles (Summary)

10.2.1 Instruction Visibility

- Instructions must reach **PoU (L2)**.
- L1I must be invalidated.
- Front-end pipeline must be flushed.

Sequence:

```
DC    CVAU
IC    IVAU
DSB   ISH
ISB
```

10.2.2 Data Visibility

- Data must reach **PoC** for DMA or other cores.
- Dirty lines must be written back.
- Snoop and FEQ must complete.

Sequence:

```
DC    CVAC
DSB   ISH
```

10.2.3 DMA → CPU

- CPU must invalidate stale lines.
- CPU must refetch from PoC.

Sequence:

```
DC    CIVAC
DSB   ISH
```

10.2.4 MMIO

- Device-nGnRnE
- No caching
- No DC/IC
- Barriers only

Sequence:

```
STR/LDR
DSB SY
```

10.3 Instruction-Related Patterns

10.3.1 Self-Modifying Code / JIT Engines

```
DC   CVAU, xN
IC   IVAU, xN
DSB  ISH
ISB
```

10.3.2 Bootloader Patching Code

```
DC   CVAU, patch_region
IC   IVAU, patch_region
DSB  ISH
ISB
```

10.3.3 Hypervisor Installing Guest Trampolines

```
DC   CVAU, guest_trampoline
IC   IVAU, guest_trampoline
DSB  ISH
ISB
```

10.3.4 Kernel Modifying Exception Vectors

```
DC   CVAU, vector_base
IC   IVAU, vector_base
DSB  ISH
ISB
```

10.3.5 CPU0 Preparing Code for CPU1 (SMP Bring-Up)

CPU0:

```
DC   CVAU, bootstrap
IC   IVAU, bootstrap
DSB  ISH
ISB
```

CPU1:

```
ISB
```

10.4 Data-Related Patterns

10.4.1 CPU → DMA (Device Reads CPU Data)

```
DC   CVAC, buffer
DSB  ISH
```

10.4.2 DMA → CPU (Device Writes Data)

```
DC   CIVAC, buffer
DSB  ISH
```

10.4.3 CPU → Other Core (Inter-Core Data Visibility)

CPU0:

```
DC   CVAC, shared_data
DSB  ISH
```

CPU1:

```
DSB  ISH
```

10.4.4 CPU → Other Cluster

```
DC   CVAC, shared_data
DSB  ISH
```

10.4.5 Zeroing Memory Efficiently

```
DC ZVA, xN
```

Allocates a zeroed line in L1D.

10.5 MMIO / Device Memory Patterns

10.5.1 Writing Device Registers

```
STR wX, [MMIO]
DSB SY
```

10.5.2 Reading Device Registers

```
DSB SY
LDR wX, [MMIO]
```

10.5.3 Writing DMA Descriptors

```
STR descriptor
DSB SY
STR start_register
```

```
DSB SY
```

10.5.4 Reading DMA Completion Status

```
DSB SY
LDR status
```

10.6 SMP Patterns

10.6.1 CPU0 → CPU1 Instruction Visibility

CPU0:

```
DC   CVAU
IC   IVAU
DSB  ISH
ISB
```

CPU1:

```
ISB
```

10.6.2 CPU0 → CPU1 Data Visibility

CPU0:

```
DC   CVAC
DSB  ISH
```

CPU1:

```
DSB  ISH
```

10.6.3 Secondary Core Boot Sequence

CPU0:

```
DC   CVAU, bootstrap
IC   IVAU, bootstrap
DSB  ISH
ISB
```

CPU1:

```
ISB
```

10.7 Hypervisor / Kernel Patterns

10.7.1 Installing Stage-2 Page Tables

```
DC   CVAC, page_tables
DSB  ISH
```

10.7.2 Modifying Stage-1 Page Tables

```
DC   CVAU, page_tables
IC   IVAU, page_tables
DSB  ISH
ISB
```

10.7.3 Updating EL2 Vectors

```
DC   CVAU, el2_vectors
IC   IVAU, el2_vectors
DSB  ISH
```

```
ISB
```

10.7.4 Updating EL1 Vectors

```
DC   CVAU, el1_vectors
IC   IVAU, el1_vectors
DSB  ISH
ISB
```

10.8 Bootloader Patterns

10.8.1 MMU Enable Sequence

```
Configure MAIR
Configure TCR
Install TTBR tables
SCTLR_EL1.M = 1
```

10.8.2 Cache Enable Sequence

```
SCTLR_EL1.C = 1
SCTLR_EL1.I = 1
```

10.8.3 Patching Boot Code

```
DC   CVAU
IC   IVAU
DSB  ISH
ISB
```

10.9 Complete Cookbook Table

Scenario	Required Sequence	Visibility Domain
Modify instructions	CVAU → IVAU → DSB → ISB	PoU
CPU → DMA	CVAC → DSB	PoC
DMA → CPU	CIVAC → DSB	PoC
CPU → CPU (data)	CVAC → DSB	PoC
CPU → CPU (instructions)	CVAU → IVAU → DSB → ISB	PoU
MMIO write	STR → DSB SY	PoC
MMIO read	DSB SY → LDR	PoC
Secondary core boot	CVAU → IVAU → DSB → ISB	PoU
Zero memory	ZVA	L1D

10.10 Exercises (Technical Reference)

Exercise 10.1 — Instruction Visibility

Explain why ISB is required after IC IVAU.

Exercise 10.2 — DMA Read Path

Describe why DC CVAC must be executed before DMA reads CPU-generated data.

Exercise 10.3 — DMA Write Path

Explain why DC CIVAC must be executed after DMA writes to memory.

Exercise 10.4 — MMIO Ordering

Describe why DSB SY is required after writing device registers.

Exercise 10.5 — SMP Bring-Up

Explain why CPU1 must execute ISB after CPU0 prepares bootstrap code.